

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
«ДНІПРОВСЬКА ПОЛІТЕХНІКА»**



**ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
Кафедра системного аналізу та управління**

**ПРАКТИЧНА РОБОТА №2
з дисципліни
«Аналіз програмного забезпечення»**

Виконав:
студент групи 124-24ск-1
Москаленко Олексій Миколайович
(П.І.Б.)

Прийняв:
Доцент каф. САУ.
Мінєєв Олександр Сергійович
Асистент САУ
Шевченко Юлія Олександрівна
(П.І.Б.)

**Дніпро
2026**

ПРАКТИЧНА РОБОТА №2

Тема: Створення і налаштування профілю у системі Git.

Мета: закріпити теоретичні знання і розвинути практичні навички у використанні системи контролю версій Git.

Завдання:

Створити власний репозиторій в GitHub. В подальшому усі результати своїх практичних робіт необхідно буде завантажувати у цей репозиторій. В репозиторії створити для кожної практичної роботи окрему папку і розмістити в ній звіт. Якщо ви ще не маєте обліковий запис, то в професійному світі запис повинен виглядати, як поєднання першої літери ім'я та прізвище в повному виді. Наприклад, Микола Єфремов- «myefremov», а Тарас Шевченко- «tshevchenko». Посилання на репозиторій надаєте викладачеві.

Виконання завдання:

Реєстрація та налаштування профілю:

Створено обліковий запис на вебсайті GitHub, ім'я якого сформовано за академічним стандартом: «omoskalenko-m»).

<https://github.com/omoskalenko-m>

Створення репозиторію:

Перейшовши до особистого кабінету на GitHub, натиснуто кнопку «New» для створення нового репозиторію.

У формі створення вказано ім'я проєкту: apz.

Встановлено рівень видимості Public (публічний).

Завершено створення натисканням кнопки «Create repository».

1 General

Owner * Repository name *

omoskalenko-m / apz

apz is available.

Great repository names are short and memorable. How about [reimagined-memory?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility * Choose who can see and commit to this repository

Public

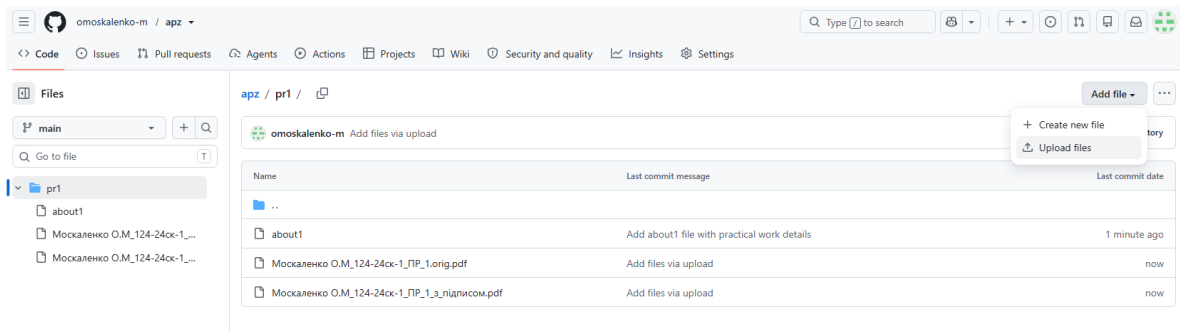
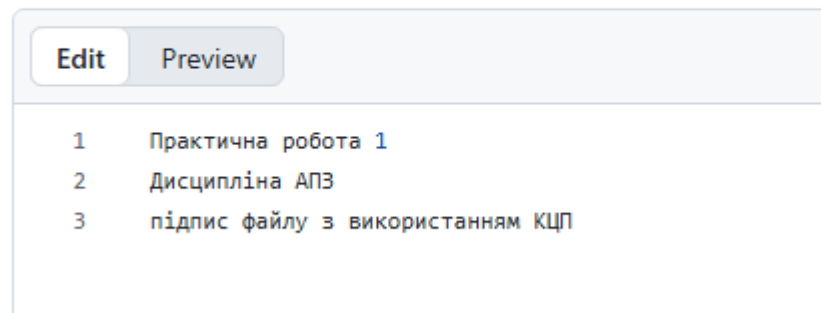
Організація структури проєкту та завантаження звітів:

Для організації звітів за кожною роботою сформовано окремі директорії (папки).

Завантажено звіт до папки першої практичної роботи (pr1/about1) за допомогою функції «Upload files» та зафіксовано зміни (коміт).

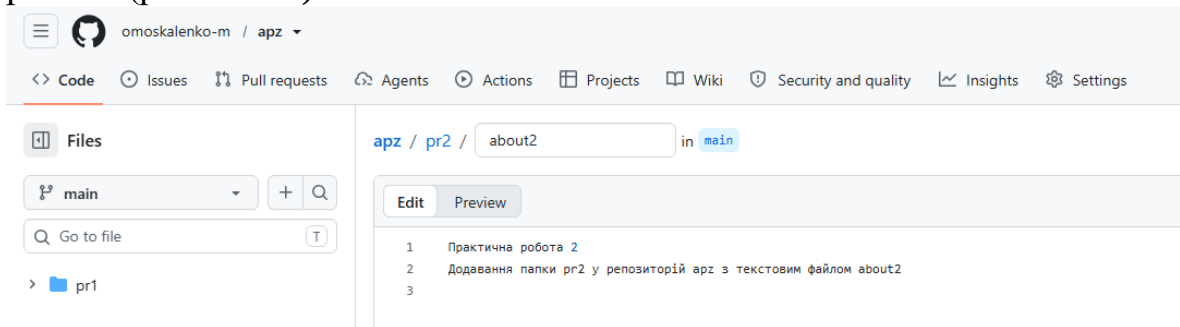
Завантаження файлів у папку практичної роботи №1

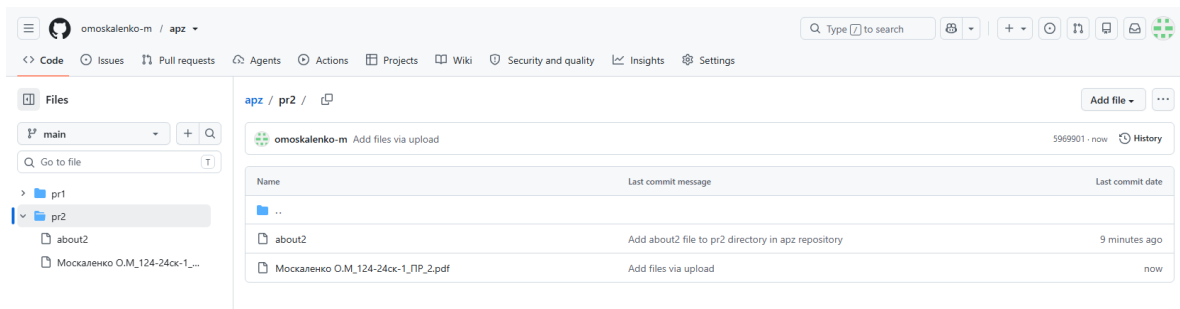
apz / pr1 / about1 in main



Завантаження файлів у папку для практичної роботи 2

За аналогічним алгоритмом завантажено звіт до папки другої практичної роботи (pr2/about2).





Висновок:

Під час виконання практичної роботи було закріплено теоретичні знання про роботу із сучасними системами контролю версій. Успішно налаштовано обліковий запис і створено публічний репозиторій `apz` на платформі GitHub. Було опановано навички організації структури проєкту у вебсервісі, створення тематичних каталогів і завантаження файлів звітів.

Контрольні питання:

1. Що таке контроль версій Git?

Git - це розподілена система керування версіями (Distributed Version Control System - DVCS), призначена для фіксації та відстеження змін у файлах комп'ютерного коду під час розробки програмного забезпечення.

Вона дозволяє кільком розробникам одночасно працювати над одним проєктом, не перезаписуючи зміни один одного, а також зберігає повну історію кожного редагування (хто, коли та що саме змінив). У разі потреби Git дає змогу безпечно повернути проєкт до будь-якої попередньої збереженої точки (коміту).

2. Що таке репозиторій у Git?

Репозиторій (repository, або «репо») - це віртуальне сховище вашого проєкту, яке містить:

- усі робочі файли та папки проєкту;
- приховану службову директорію `.git`, де зберігається вся база даних історії змін, конфігурація проєкту, гілки (branches) та покажчики комітів.

Репозиторії бувають двох типів:

- Локальний: знаходиться безпосередньо на комп'ютері розробника.
- Віддалений (remote): розміщується на сервері або хмарній платформі (наприклад, GitHub, GitLab, Bitbucket) для синхронізації та спільної роботи команди.

3. Які переваги використання Git?

- Розподілена архітектура: кожен розробник має повну копію історії репозиторію локально, тому більшість операцій (коміти, перегляд логів, створення гілок) виконуються миттєво і не потребують підключення до мережі.
- Швидкість і продуктивність: Git оптимізований для швидкої роботи навіть із великими проєктами.
- Ефективне розгалуження (branching): створення нових гілок для ізольованої розробки функціоналу (features) чи виправлення помилок (bugfixes) відбувається миттєво, а подальше злиття (merge) максимально автоматизоване.
- Цілісність даних: кожен об'єкт у Git (файл, дерево, коміт) має унікальний криптографічний хеш (SHA-1/SHA-256), що виключає можливість непомітного пошкодження або підміни даних.
- Зручність командної взаємодії: інтеграція з механізмами Pull/Merge Requests полегшує рев'ю коду та аналіз його якості.

4. Як можна створити репозиторій у Git?

Існує два основні способи:

- Ініціалізація нового локального репозиторію з нуля:

У терміналі перейти в папку вашого проєкту і виконати команду:

git init

Це створить службову папку `.git` у поточній директорії.

- Клонування існуючого віддаленого репозиторію:

Скопіювати вже наявний проєкт із сервера (GitHub/GitLab тощо) на локальний комп'ютер:

git clone

(Також створити репозиторій можна безпосередньо через вебінтерфейс хмарних сервісів - наприклад, натиснувши кнопку «New repository» на GitHub, після чого клонувати його на свій ПК).

5. Чи можна надати доступ до репозиторію сторонній людині?

Так, можна. Спосіб надання доступу залежить від типу репозиторію та платформи хостингу (GitHub, GitLab тощо):

- Зробити репозиторій публічним (Public): будь-який користувач мережі зможе переглядати та завантажувати (clone/fork) код.

- Додати співавтора (Collaborator): якщо репозиторій приватний, у налаштуваннях платформи (Settings → Collaborators / Access Management) можна запросити конкретну людину за її нікнеймом або email, надавши права на читання (Read), запис (Write) чи адміністрування.
- Через команди / організації: на корпоративних та групових акаунтах доступ налаштовується централізовано через ролі для команд і груп користувачів.
- Механізм Fork & Pull Request: стороння людина може зробити копію (fork) публічного проекту до свого профілю, внести зміни та запропонувати їх власнику оригінального репозиторію через Pull Request.